FINTECHATHON 2026 / INTERNATIONAL TRACK

Security Self-Assessment Report

Sentrion | SME Invoice Operations & Financial Risk MVP

Project	Sentrion — SME Invoice Operations & Financial Risk MVP

Assessment position

Suitable for a controlled local demonstration with synthetic data. This report does not approve the build for public exposure, multi-business use, or real supplier banking data.

Prepared by the Cing Cong Fan team for FinTechathon 2026 — Topic E: SME Finance Copilot.

Assessment details

- **Build reviewed:** Sentrion MVP
- **Assessment type:** Team self-assessment; not an independent audit or certification

1. What we reviewed

The review used `Sentrion_MVP(6).zip`, which identifies the application as version 0.2.6. The archive has no Git metadata, so this report cannot identify a commit hash. The review date is 20 October 2026.

We inspected the FastAPI routes, application configuration, authentication helpers, SQLite schema and migrations, invoice/PO processing, extraction and optional LLM services, vendor-resolution rules, risk assessment, document access routes, audit helpers, templates, and test suite. We also ran these commands against an extracted copy of the archive:

```
python -m pytest -q
python -m compileall -q app tools_*.py tests
```

The test suite completed with **71 passed in 7.02 seconds**. Python compilation also completed successfully. Tests ran in the supplied local test setup, which uses a temporary data directory and database and leaves external AI disabled.

This was a source review and functional regression run, not a penetration test. We did not perform dependency vulnerability scanning, an independent privacy or legal review, a live Gemini/OpenAI test, an assessment of a hosting platform, or any bank-account verification. Where this report says a control was “not found” or “not verified,” it means we could not substantiate it in the supplied build; it is not a claim about every possible deployment configuration.

2. The operating boundary matters

Sentrion runs as a local FastAPI application. SQLite stores application records, and invoice, PO, and vendor-evidence files are stored under the configured local data directory. The roles are Finance, Supervisor, and Admin, but the application is still one shared workspace. We did not identify tenant-level or organisation-level data partitioning in the reviewed routes, so authenticated users can access the shared records and protected documents made available by the application.

External structured extraction is optional and disabled by default. If a team member enables it and supplies a valid API key, the application sends extracted document text—up to 45,000 characters on the reviewed request path—to the configured OpenAI-compatible Chat Completions endpoint. The prompt treats document text as untrusted input, the returned structure is restricted and type-checked, and trace records omit API keys, raw text, and provider response bodies. These measures reduce some risks, but they do not make model output reliable by default. Automated tests use mocked provider responses; live provider behavior was not evaluated.

Payments are simulated. An eligible GREEN invoice can enter the mock-payment workflow, but no funds move and no bank API is called. Similarly, a SHA-256 hash can help identify whether stored bytes differ from the recorded version; it cannot prove that a document is genuine, that a bank account belongs to a named supplier, or that the database audit history is immutable.

3. Controls we found in the source

We used three practical labels while reviewing the controls: **implemented** means the mechanism is present in the code; **test-supported** means one or more supplied tests assert related behaviour; and **partial / not verified** means we found a gap or could not establish enough evidence. These labels describe this review only. They are not a certification.

Control	What we found	Current limitation
Password storage	`app/security.py` uses PBKDF2-HMAC-SHA256 with a random 16-byte salt and 310,000 iterations; digest comparison uses `hmac.compare_digest`.	The login route does not throttle repeated failed attempts.
Session cookie	Starlette `SessionMiddleware`; cookie name `sentrion_session`, 8-hour maximum age and `SameSite=Lax`.	HTTPS-only is configurable but defaults to false, and a known local-demo secret fallback exists when `SENTRION_SESSION_SECRET` is missing.
CSRF	State-changing form routes call CSRF validation. Tokens are generated using `secrets.token_urlsafe(32)` and compared with `hmac.compare_digest`.	We found related workflow coverage, but dedicated negative tests for missing and invalid CSRF tokens should be expanded.
Server-side permissions	Protected action routes check the authenticated user and required role. Tests cover selected vendor and approval	There is no complete route-by-route authorisation

Control	What we found	Current limitation
	restrictions.	matrix test.
Maker-checker	Sensitive vendor and selected exception workflows prevent self-approval and require a distinct final reviewer.	The same two-person rule does not apply to every ordinary GREEN invoice approval.
Upload validation	Supported formats, configured size limits, PDF signatures and image validity are checked; some vendor evidence formats have additional checks.	No malware scanner or isolated file-analysis service was identified.
Document access	Document and evidence routes require authentication and constrain resolved paths to the configured upload directory.	All permitted records are in the same workspace; there is no tenant isolation.
Document integrity	SHA-256 hashes and size metadata are stored and compared when evidence is reviewed or downloaded.	Hashes do not make the database or audit history tamper-proof.
Vendor identity	Exact normalised tax-ID matching is the primary automatic link method for eligible verified vendors. Name-only matches remain candidates; conflicting reliable tax IDs are treated as high risk.	Bank-account ownership still needs independent evidence and human verification.
Risk and payment gate	GREEN/YELLOW/RED findings are generated from configured rules. Tests cover selected risk-gating and mock-payment paths.	Thresholds are demonstration policy, not a validated accounting standard.
Optional LLM	External AI is off by default. When enabled, the response is field-restricted and type-checked; traces omit keys, raw text and provider bodies.	Tests mock provider responses. Prompt-injection detection is not comprehensive, and live provider reliability is unknown.
Security headers	Some document/evidence responses set headers such as `Cache-Control` and `X-Content-Type-Options`; evidence responses also use a sandboxed CSP.	We found no global middleware applying a consistent security-header policy to the whole application.
Data storage and audit retention	The application uses SQLite, local files and audit events.	No configured at-rest encryption, tested backup/restore process, retention policy, or append-only external audit store was evidenced.

4. Risks that need attention

The ratings below reflect potential impact if the prototype were shared or hosted. They are qualitative judgements by the team, not measured likelihoods. We would address the HIGH items—or document an explicit exception—before any shared pilot with real business data.

ID / rating	Risk and why it matters	Recommended next step
R-01 — HIGH	**Session-secret fallback and insecure cookie default.** A missing secret can fall back to a known demo value, and secure cookies are not enabled by default. These defaults are acceptable only for a local demo.	Fail closed outside explicit local-demo mode when the secret is missing or a placeholder. Use a strong secret from protected configuration and enable secure cookies behind correctly configured HTTPS.
R-02 — HIGH	**No login throttling or lockout.** Repeated incorrect credentials return an error, but the reviewed login route does not track or limit failed attempts.	Add rate limiting or progressive delays, with tests for repeated failures and monitoring for abuse.
R-03 — HIGH	**No tenant-level data separation.** Authenticated users share the same workspace and document scope. This is not suitable for several unrelated businesses using one instance.	Keep demonstrations restricted to one trusted group, or implement organisation/tenant scoping in every record query and document route before multi-business use. Add cross-tenant negative tests.
R-04 — HIGH	**Fixed demo credentials and seed accounts.** Demo accounts are useful for local evaluation but unsafe on a publicly reachable service.	Disable demo seeding outside explicit local mode, create individual accounts securely, and rotate credentials before any shared demonstration.
R-05 — HIGH	**Local storage and mutable audit history.** SQLite and local files do not provide a demonstrated backup, retention, at-rest encryption, or immutable audit process.	Design private persistent storage, least-privilege service access, backup/restore drills, retention, and a separately controlled audit export before a hosted pilot.
R-06 — HIGH	**No independent bank-account ownership check.** The app can compare document values with vendor records, but it cannot ask a bank to confirm ownership.	Define a reviewer procedure using evidence obtained through an independent official channel. Do not infer ownership from a matching name or masked account number alone.

ID / rating	Risk and why it matters	Recommended next step
R-07 — HIGH for real-money workflows	**Not every GREEN invoice requires two-person approval.** The standard GREEN path can create a simulated payment after the risk gate, but the code does not enforce a second approver for every such invoice.	Decide the policy before adding real payments. If separation of duties is required, enforce it server-side and test attempts to bypass it.
R-08 — MEDIUM	**No global security-header policy.** Some responses set specific headers, but a consistent application-wide policy was not identified.	Add a suitable global policy and test it across pages and error responses. Apply HSTS only behind correctly configured HTTPS.
R-09 — MEDIUM	**External AI can receive business document text.** Enabling the provider sends extracted text to a third party; mock tests do not establish the provider's live behaviour or data-retention terms.	Keep the feature off unless its use is approved. Review provider terms, minimise data where practical, restrict endpoint configuration, and test with synthetic documents first.
R-10 — MEDIUM	**Prompt-injection detection is not exhaustive.** The known test case is detected, but no heuristic can guarantee recognition of every malicious or disguised instruction.	Treat all document and model output as untrusted, keep approval authority outside the model, and add an adversarial test corpus.
R-11 — MEDIUM	**Uploads are validated but not malware-scanned.** Type, signature and size checks reduce some input risk but do not detect every harmful file.	Add malware scanning and safer parser isolation before accepting documents from unknown external parties.
R-12 — MEDIUM	**Sensitive records are shared across roles.** Role checks protect privileged actions, but record-level read scoping is limited by the shared-workspace model.	Define least-privilege access for invoices, bank evidence and audit details, then test direct URLs for every role.
R-13 — MEDIUM	**No independent security test.** The regression suite is useful, but this archive contains no penetration-test report or dependency vulnerability report.	Run dependency and secret scans, expand negative authorisation tests, and obtain an independent review before shared or hosted use.
R-14 — MEDIUM	**Risk rules are prototype policy.** A correctly extracted value can still be assessed incorrectly if the configured business rule does not match the company's accounting policy.	Review the rules with finance/accounting users, version threshold changes, and reconcile decisions against labelled cases.

5. Test results and their limits

We reran the tests after extracting the supplied archive. The current result is:

- `python -m pytest -q` — **71 passed in 7.02 seconds**.
- `python -m compileall -q app tools_*.py tests` — **completed successfully**.

The seven test modules cover the following areas:

Test module	Collected tests	What it checks
`tests/test_currency.py`	8	Currency parsing, amount formatting, currency mismatch and ambiguous-currency cases.
`tests/test_db_migrations.py`	2	Forward migrations and preservation of existing records and relationships.
`tests/test_extraction_risk.py`	9	Document extraction examples, OCR-related paths, prompt-injection signal and selected risk outcomes.
`tests/test_llm_traceability.py`	9	Mock provider success/failure, disabled or missing-key behaviour, sanitised trace metadata, retries and local fallback.
`tests/test_revision_features.py`	6	Dataset-layout examples, account-holder fallback, PO total handling, PO detail and LLM-proposed totals needing review.
`tests/test_vendor_resolution.py`	10	Tax-ID matches, name-only candidates, conflicting IDs, ambiguity, PO relinking, idempotency and role restrictions.
`tests/test_web_workflows.py`	27	Login and pages, uploads, mock-only payments, risk blocking, staged approvals, vendor evidence, document integrity and audit-related workflows.

These results tell us that the tested paths behave as expected for the cases in the suite. They do **not** prove OCR accuracy across a representative real-world invoice corpus, resistance to every prompt injection, brute-force protection, complete authorisation coverage for every route, live Gemini/OpenAI reliability, production backup/restore, or hosted deployment readiness.

There is no `test_security_hardening.py` in this archive. Dedicated negative tests are still needed for invalid or missing CSRF tokens, missing production secrets, login throttling, global security headers, direct-URL access across roles, and organisation-level data separation.

6. Work we would complete before a shared pilot

We would not start by adding more AI features. The first release gate should close the deployment and access-control gaps already identified.

1. **Remove unsafe deployment defaults.** Require a strong session secret outside local-demo mode, set cookie security correctly behind HTTPS, and remove fixed demo accounts from shared environments.
2. **Define who can see each record.** Either limit the app to a single controlled demo group or implement organisation and record scopes across database queries and document delivery.
3. **Harden authentication and HTTP responses.** Add login throttling, a consistent security-header policy, and negative tests for these controls and CSRF validation.
4. **Confirm approval policy.** Decide whether all invoice approvals need a second person and enforce the chosen rule in backend logic, not just in the interface.
5. **Plan data protection operations.** Choose a persistent storage architecture, define backups and restore checks, set audit retention, and limit operating-system access to the data directory.
6. **Strengthen document and AI handling.** Add malware scanning or equivalent safe processing for untrusted uploads. Keep external AI opt-in and review the data-sharing boundary before enabling it.
7. **Get an independent review.** Run dependency and secret scans, expand route-level negative tests, test adversarial documents, and have a security reviewer challenge the assumptions in this report.

7. Overall assessment

The current Sentrion build has a sound starting point for a supervised prototype. It uses password hashing, CSRF checks, server-side role enforcement, staged review for sensitive vendor changes, deterministic risk findings, audit events and simulated-only payments. The 71 passing tests give useful evidence that selected workflows and negative paths work as intended.

That evidence should not be confused with production assurance. The local-demo secret fallback, lack of login throttling, shared data scope, mutable local storage, incomplete global header policy and absence of independent bank verification remain material gaps. The app should stay in an isolated environment with synthetic data until those risks are addressed and retested.

Disposition: acceptable for a controlled local demonstration using synthetic data. This self-assessment does not approve the build for public internet exposure, multi-business use, real supplier banking information, or real payments. It is not legal advice, a compliance attestation, or an independent security audit.

Appendix A. Source-to-assessment traceability

Assessment point	Source reviewed	How it was checked
Password hashing	`app/security.py` (`hash_password`, `verify_password`); `app/seed.py`	Source inspection and login workflow tests.
Session cookie and secret defaults	`app/main.py` (`SessionMiddleware`); `app/config.py`	Source inspection; deployment default reviewed.
CSRF controls	`app/security.py` (`ensure_csrf`, `verify_csrf`); POST handlers in `app/main.py`	Source inspection; related tests reviewed. Dedicated invalid-token coverage remains incomplete.
Role permissions and approval separation	Protected invoice, approval, vendor and PO routes in `app/main.py`	Source inspection and selected workflow tests.
Vendor identity and PO relinking	`app/services/vendor_resolution.py`; `tests/test_vendor_resolution.py`	Source inspection and match/candidate/conflict/relink test cases.
Risk classification and payment gate	`app/services/risk.py`; `tests/test_extraction_risk.py`; `tests/test_web_workflows.py`	Source inspection and selected GREEN/YELLOW/RED workflow tests.
Upload and protected document access	Upload helpers and document/evidence routes in `app/main.py`	Source inspection and upload/integrity workflow tests; no malware scanner found.

Assessment point	Source reviewed	How it was checked
Optional LLM boundary and trace	`app/services/llm_extraction.py`; `tests/test_llm_traceability.py`	Source inspection and mocked provider tests; live provider not tested.
Data persistence and audit history	`app/db.py`; `app/services/audit.py`	SQLite and local-file implementation inspected; immutable external audit storage not found.
Test execution	Seven modules under `tests/`	`pytest`: 71 passed in 7.02 seconds; `compileall`: exit code 0.